SVN Code Drop Upload Tool

Brainstorm document, revised after the screen mockups and path-based baselines | 8 October 2026

1. Purpose

A web application that takes a code drop folder (for example Codedrop10), compares it with the latest previous code drop already in SVN, produces Excel review reports, checks the configs for dummy values, and uploads to SVN only after the user approves. The goal is to catch release mistakes (missing version bumps, wrong configs, leftover test values) before they reach SVN.

2. Folder structure and naming

```
Codedrop10/
  Binaries/TMACServer/TMACServer.exe
  Configs/TMACServer/app/template.json
```

- **Binaries** holds dlls and exes in component subfolders (for example TMACServer). **Configs** holds mainly json files, in component and sub-subfolders (for example TMACServer/app).
- Files are matched between code drops by **subfolder path and file name**. A file found under a different subfolder is reported as moved, not as removed plus new.
- Code drops are numbered by the naming convention (Codedrop##), which gives the order in which earlier versions are searched. Section 3 explains how baselines are chosen.
- A code drop is **not required to contain both** Binaries and Configs, or every component. The pre-check only requires at least one of them; an area that is missing in the current code drop is skipped and noted in the reports.

3. Baseline resolution per folder path

A code drop does not always contain every component. Codedrop10 may contain TMACServer and TMACProxy, while the latest earlier version of TMACServer is in Codedrop07 and the latest earlier version of TMACProxy is in Codedrop08. A single "previous code drop" would therefore be wrong, so the baseline is resolved **per component**. The same applies to areas: an earlier code drop may contain only Configs or only Binaries, so the search is done for each area and component **path** (for example Binaries/TMACServer, Configs/TMACServer) and the baseline is always found from the folder and file structure path.

- **Rule:** for each path of the form Area/Component (for example Binaries/TMACServer), search earlier code drops from the newest downward (Codedrop09, 08, 07 ...) and pick the first one that actually contains **that path**. A code drop that lacks the path is skipped for that path, even if it contains other areas or components.
- Binaries and Configs are resolved **separately**, so the same component can have different baselines in each area. Example: Codedrop09 contains only Configs/TMACServer, so it is the baseline for those configs, but the search for Binaries/TMACServer continues past it to Codedrop08 and Codedrop07.
- A component found in no earlier code drop is treated as new: it has no baseline and all its files are listed as New.
- The baseline and its SVN revision are shown on the Select screen, and in every group header on the Compare and Config diff screens. The user can change any baseline manually.
- Every Excel sheet states the baseline path of each row, so a report is clear on its own.
- Within a component, files are matched by relative path and file name.

- Listing many SVN folders is slow, so keep a cached index of which code drop contains which component (with SVN revision) and update it after each upload.

Path to compare	Baseline	Why
Binaries/TMACServer	Codedrop07	Codedrop09 has no Binaries; Codedrop08 has no Binaries/TMACServer
Configs/TMACServer	Codedrop09	Codedrop09 has only Configs/TMACServer, which is enough for this path
Binaries/TMACProxy	Codedrop08	Latest earlier drop containing Binaries/TMACProxy
Configs/TMACProxy	Codedrop08	Latest earlier drop containing Configs/TMACProxy
Binaries/TMACAgent	None	Not found in any earlier drop: new component

4. User flow (five screens)

Screen	What the user does / sees
1. Select	Enters the SVN path and folder name, drops the folder. The app validates every config json file; any failure is listed with file name and line number and blocks the comparison until the Configs folder is re-uploaded. The screen also shows the detected structure and pre-checks: at least one of Binaries or Configs found, name matches the pattern, and the baseline code drop found for each area and component path (for example Binaries/TMACServer in Codedrop07, Configs/TMACServer in Codedrop09).
2. Compare (binaries)	Banner shows the current folder and one baseline chip per component. Binaries are grouped by component subfolder; each group header shows its own pair of paths (for example Codedrop10 vs Codedrop07 for TMACServer, Codedrop10 vs Codedrop08 for TMACProxy) and a Change baseline button. Columns: file, baseline version, current version, status.
3. Config diff	List of json files on the left, each tagged with its baseline (for example vs Codedrop07). Right side shows the full current and baseline paths and a before/after diff. A button captures the diff as a screenshot for Configs.xlsx.
4. Dummy scan	Table of placeholder values found in the current configs: file, key path, value, matched pattern, severity.
5. Approve	Dummy-value re-check after correction (each re-scan requires re-uploading the Configs folder), upload checklist, report downloads, commit message, and the Confirm and upload button, which stays disabled until the checks pass.
5b. JSON validation failed	Shown when an uploaded or re-uploaded Configs folder contains invalid json. Red banner, counts (files checked, valid, failed), an errors table with file, line, column and message, and the offending line shown in context. The checklist shows the dummy scan and warnings as not run. Actions: download the error list or re-upload the Configs folder; upload stays disabled.

5. Binaries comparison

- Product version is read from the exe or dll file properties (Details tab). Windows hosting makes this simplest (.NET FileVersionInfo); on Linux use a PE reader such as pefile.
- **Statuses:** Changed, Unchanged, New, Removed, Moved, and **Hash changed, same version** (a rebuild without a version bump, flagged as a likely mistake).
- Also flag a current version **lower** than the baseline (possible downgrade).
- Dlls are compared the same way as exes, even though only exes were mentioned.
- Missing version resource: show N/A rather than failing.

Binaries.xlsx

Sheet	Columns
Baselines	One row per area and component path (for example Binaries/TMACServer, Configs/TMACServer): baseline code drop, baseline path, SVN revision, note (for example "New component, no earlier version"). The same component can have different baseline code drops for Binaries and Configs.
Binary Comparison	Subfolder, File, Baseline folder path (differs per component), Baseline version, Current version, Status. Current version column highlighted.
Version History	One row per exe; one column per code drop (Codedrop06 to Codedrop10). A dash means the component is not in that code drop; the baseline cell is highlighted green and the current column is highlighted.

Fetching versions from every old code drop is slow, so cache the version of each file per code drop (small database or json index) and read SVN only for new drops.

6. Config comparison

- Each json file is compared with the same relative path in the baseline code drop of its own component (for example TMACServer/app/template.json against Codedrop07, TMACProxy/proxy.json against Codedrop08).
- Use a **semantic json diff** (key level: added, removed, changed) as well as the visual text diff, so formatting and key order do not create noise.
- The diff is rendered as an image (headless browser or direct drawing) and embedded in Excel. Because screenshots are not searchable, each sheet also lists the changed keys with previous and current values.
- New and removed json files and a byte order mark need explicit handling; malformed json is caught earlier by the JSON validation step (next subsection).
- Mask values of keys such as password, secret, token and connection strings in screenshots, because the Excel report would otherwise leak real secrets.

Configs.xlsx

Sheet	Content
Summary	One row per config file: baseline path, status (changed, new, removed, unchanged) and number of keys changed.
One sheet per json file	Named like TMACServer_app_template.json (Excel limits sheet names to 31 characters). Header shows the current path against the baseline path. Diff screenshot at the top, then a table of key path, previous value, current value, change.
Dummy Values	Config file, key path, value found, matched pattern, severity. The header shows the version of the rules file used.

JSON validation before any comparison

- Every uploaded config json file is parsed with a strict json parser **right after upload**, before the baseline diff or the dummy scan, because an invalid file cannot be diffed or scanned reliably.
- If a file fails, the app shows the **file name (relative path), line number and column** plus a short message, for example "TMACServer/app/appsettings.json, line 14, column 5". Showing the offending line text is a useful extra.
- A failed validation **blocks** the comparison and the upload. The user fixes the files and re-uploads the Configs folder, and the validation runs again.

- The result is an item in the upload checklist ("All config JSON files valid"), recorded in the audit log together with the upload time and folder hash.
- A parser usually reports only the first error per file, so after a fix the next error in the same file may appear on the next upload. Showing every file with an error at once keeps the number of re-uploads down.
- Decide how strict to be about common non-standard content: comments, trailing commas, a byte order mark, and duplicate keys. Strict json rejects comments and trailing commas; duplicate keys are valid for most parsers but are almost always a mistake, so report them at least as a warning.
- **Failure screen:** a red banner states how many files failed and that the dummy scan, comparison and upload are blocked. Counts show files checked, valid and failed. The errors table lists file, line, column and message; selecting a row shows the offending line with a few lines around it, highlighted. Checklist items that depend on valid json show "Not run". The user can download the error list or re-upload the Configs folder.
- Non-json config files (xml, ini and so on) are not covered by this check unless an equivalent validator is added.

File	Line	Column	Error
TMACServer/app/appsettings.json	14	5	Expected comma or closing brace after property value
TMACProxy/routes.json	9	22	Trailing comma is not allowed

7. Dummy value detection and the re-check gate

- Starter patterns: tetherfi, test, password, abc, xxxx. The list is kept in a **config file** (see below) and supports contains, whole word, exact and regex matching.
- Scan values, optionally keys, and report the full key path (for example Db.Password).
- Likely extra patterns: changeme, localhost, 127.0.0.1, TODO, sample, dummy, example.com, empty required values.
- Reduce false positives (for example "test" inside "latest") with whole-word matching and an allowlist per key.
- Severity: **Blocker** stops the upload; **Warning** needs an explicit acknowledgment.

How the scanner decides

- **Pattern rules:** every json value is checked against the configured patterns (tetherfi, test, password, abc, xxxx and so on).
- **Generic placeholder checks:** repeated characters (xxxx-xxxx, 0000), template markers (CHANGE_ME, TODO, {{host}}), common sample values (localhost, 127.0.0.1, example.com) and empty values for keys that need one.
- **Key-name context:** the full key path is read as well as the value. A weak value under a key such as Db.Password is a blocker; localhost under an endpoint key is a warning.
- **Allowlist:** specific file and key path combinations can be marked as acceptable, so known false positives do not keep reappearing.

Dummy value rules live in a config file

- The dummy values are **not hard-coded**. They are stored in a settings config file (json) that the app reads at the start of every scan, so changing the list does not need a code change or a redeploy.
- Each rule has a pattern, a match type (contains, wholeWord, exact, regex), a severity (Blocker or Warning), and optionally an enabled flag and a component scope.

- The same file holds the allowlist and global settings such as ignoreCase and whether keys are scanned as well as values.
- The file is kept under version control (or in the app deploy folder with a change log), so every change to the rules can be reviewed and traced.
- Each scan records which version or hash of the rules file it used, and the Dummy Values sheet in Configs.xlsx shows it, so a report can be reproduced later.
- The file is validated when it is read. An invalid regex or an unknown match type stops the scan with a clear error instead of silently scanning with no rules.
- Only admins can edit the file. An admin settings screen for editing rules is an optional later addition; a config file is enough for the first version.

Example of the rules file (illustrative):

```
{
  "ignoreCase": true,
  "rules": [
    {"pattern": "tetherfi", "match": "contains", "severity": "Blocker"},
    {"pattern": "test", "match": "wholeWord", "severity": "Blocker"},
    {"pattern": "password", "match": "contains", "severity": "Blocker"},
    {"pattern": "abc", "match": "wholeWord", "severity": "Warning"},
    {"pattern": "x{3,}", "match": "regex", "severity": "Warning"}
  ],
  "allowlist": [
    {"file": "TMACServer/app/template.json", "keyPath": "Logging.Category",
     "pattern": "test", "reason": "Category name"}
  ]
}
```

Re-check after correction (Approve screen)

- After the user fixes configs, **Re-scan configs** first asks them to re-upload the Configs folder, then runs the whole scan again, not only the earlier findings, so a fix that introduces a new dummy value is caught.
- The table shows each earlier finding with original value, current value and status: Fixed, Still present, New finding, or Warning.
- Confirm and upload stays **disabled** while any blocker remains. Warnings must be acknowledged.
- Configs.xlsx is regenerated after each re-scan so the report always matches what will be uploaded.
- Upload checklist: structure valid, config JSON files valid, binary versions reviewed, re-scan has no blockers, warnings acknowledged.

Every re-scan starts with a re-upload of the Configs folder

- Clicking **Re-scan configs** opens an upload prompt for the corrected Configs folder, which is JSON-validated first. The app never re-scans the copy uploaded earlier, so the scan always reflects the files the user actually fixed.
- After the upload the app replaces its working copy of Configs, runs the full dummy scan, regenerates the json diffs against each component baseline, and rebuilds Configs.xlsx.
- The Approve screen shows the time and a folder hash of the last scanned upload. At commit time the app checks that the files being committed still match that hash, so only the scanned and clean copy reaches SVN.
- Earlier uploads and their scan results are kept as scan history for the audit log.
- Binaries are not re-uploaded unless the user chooses to replace them as well.

8. Approval and SVN upload

- The confirmation dialog names the code drop and the SVN path, and requires a commit message.
- Record who approved, when, the SVN revision created, and the report files (audit log). Optional: two-person approval for production.
- SVN access through the svn command line (simple) or a library such as SharpSvn or pysvn. Needed operations: list, export or cat for the baseline, import or commit for the upload.
- Decide on service account versus the user own credentials, handle commit failure, a target folder that already exists, and two users uploading at once.
- Optionally verify checksums after the commit and store the reports next to the code drop.

9. Architecture options

Layer	Option A (Windows / .NET)	Option B (Python)
Backend	ASP.NET Core Web API	FastAPI or Flask
Frontend	Blazor, React or Razor	React or Vue
Excel	ClosedXML or EPPlus	openpyxl or XlsxWriter
SVN	SharpSvn or svn CLI	pysvn or svn CLI
Diff screenshots	Playwright .NET or System.Drawing	Playwright or Pillow
Exe version	FileVersionInfo (native)	pefile

Run comparison as a background job with progress shown in the UI, since large drops take time.

10. Risks and edge cases

- First release with no previous folder, or an ambiguous naming convention.
- Very large binaries and slow uploads from the browser; consider server-side paths or zip upload.
- Secrets leaking into reports and temp storage; clean temp files after each run.
- SVN downtime mid-process; partial commits.
- Non-json files in Configs (xml, config, ini): decide on a plain text diff fallback.
- A baseline picked automatically is wrong; this is why every baseline is always visible and changeable.
- A component missing from many code drops makes the baseline search long; use the cached component index.
- A config with a json syntax error cannot be diffed or scanned, so validation must always run first and block the flow.
- A code drop that has only some areas (for example only Configs/TMACServer) must not stop or shorten the search for the other areas.
- Re-uploading configs after every fix is slower for users; keep the upload quick (drag and drop or zip) and remember the last used SVN path.

11. Phased plan

Phase	Scope
1. Core compare	Structure validation, SVN listing and per-path (area and component) baseline detection, exe and dll version comparison, Binaries.xlsx.
2. Configs	Json validation with file name and line number errors, json diff, screenshots, dummy scan driven by the rules config file, Configs.xlsx.

Phase	Scope
3. Approval and upload	Re-scan gate with config re-upload, approval screen, SVN commit, audit log.
4. Polish	Version history cache, admin settings screen for the dummy rules, notifications, dashboard of past uploads, dry-run mode.

12. Open questions

- Exact folder naming convention, and how earlier code drops are ordered (name number, date or SVN revision).
- Windows or Linux hosting.
- Should dummy blockers ever be overridable by an authorised approver?
- How many earlier code drops should the version history show?
- Are config screenshots mandatory, or is a colour-coded text diff in Excel acceptable?
- Which SVN account performs the commit?
- Are json comments, trailing commas and duplicate keys accepted, warned about, or treated as errors?
- Who may edit the dummy-value rules file, and how are changes reviewed?
- If the current code drop has only Binaries or only Configs, should the app warn that the other area is missing?
- Should warning acknowledgments carry over after a re-upload when the flagged value has not changed?
- Can a component be renamed or split between code drops, and how should it be matched?